How Laxu uses Perpl

Laxu runs on Monad testnet (chain ID 10143) and trades on Perpl, the perpetual futures exchange Laxu integrates with. Every number below comes from a transaction, a script output, a test, or a file in this repository. Each claim is labelled with one of three levels:

- **Proven:** a transaction or a script output shows it.
- **Working:** built and used by the app, without a separate recorded proof.
- **Not exercised:** built, but not yet run end to end.

Contents

1. Summary
2. The angle: verifiable positions for social trading
3. How Laxu uses Perpl's API (reference)
4. Reliable execution
5. Risk management and profitability
6. Fit with the judging criteria
7. Real on-chain activity
8. Screenshots
9. Reproduce it
10. Status, limits and open items

1. Summary

- **What Laxu does on Perpl.** A user pays AUSD, and Laxu opens a real Perpl position for them. It deposits into a Perpl account, sends a signed IOC market order over Perpl's trading WebSocket, then mints a token on Monad that represents exactly that position. The token prices itself from Perpl's mark, read from Perpl's exchange contract.
- **The angle: idea 04, social trading.** Every Laxu position is a verifiable token. Anyone can follow a trader by buying a slice of that exact position, the creator earns 2% of each buy-in, and holders can borrow against their slice. Laxu does not implement options. It is not a structured-products entry (see §2).
- **The four criteria in one line each.**
  - *Reliable execution:* a resumable open flow, and an order whose result is unknown is decided on-chain, never sent twice.
  - *Risk management:* leverage caps per market, entry checked against Perpl on-chain, tiered LTV, and liquidation that never pauses.

    - *Profitability:* Laxu is infrastructure, not a strategy. Our own test trades are reported with real fees and funding.
    - *On-chain activity:* 69 orders, 37 fills, 4 position tokens, 4 lending pools, 7 borrows and 1 repay, all counted by a script.
- **The honest gap.** No strategy bot was run, and our test trades lost $0.62 in total: $0.42 from price moves and $0.19 in fees (§5.2). Laxu is **not** a trading bot and does not pick trades.
- **Evidence index:** transactions in §7; the full run log is `docs/e2e-run.md`; exchange behaviour we measured is in `docs/perpl-findings.md`.

2. The angle: verifiable positions for social trading

The problem. On a perp exchange, a follower can only copy a trader by watching them and placing their own orders, late and at a different price. A trader's results are claims on a screenshot unless someone can check them against the exchange.

The mechanism.

1. **Perpl account.** Each open trade runs on its own Perpl account ("slot"), owned by a Laxu slot wallet.
2. **Position.** The entry is a real Perpl IOC order. Perpl forwards it on-chain, so the fill is a Monad transaction to Perpl's Exchange.
3. **`PositionToken`.** An ERC-20 / ERC-7540 clone is created for that position. `createPosition` reverts unless Perpl's on-chain position has the same direction and size, with an entry within 0.5% (`PositionToken.sol:384-391`).
4. **NAV from Perpl's mark.** `totalAssets()` uses Perpl's mark, read on every valuation through `PerplReader`. The operator cannot set a price (test: `Venue.js:291`).
5. **Buy-ins.** A follower buys a slice at NAV. Laxu grows the same Perpl position by that amount, and 2% of the buy-in goes to the creator (`PositionToken.sol:54`, `:546`).
6. **Isolated lending pool.** One pool per token. Holders, creator or followers, can borrow AUSD against their shares.

Social trading capabilities

Feature	Built?	Where
Verifiable position per trader	**Yes**	`createPosition` checks against Perpl; `venueDrift()` compares at any time (`PositionToken.sol:992`)
Follow by buying in	**Yes** (proven, e2e step 5)	`requestDeposit` → backend grows the Perpl position → `DepositFulfilled`

Feature	Built?	Where
Creator fee (2% of each buy-in)	**Yes** (proven: $0.40 on a $20 buy-in)	`BUY_IN_FEE_BPS = 200`, `PositionToken.sol:54`
Per-holder stop-loss / take-profit	**Yes** (proven, e2e step 7)	`setTriggers` / `executeTrigger`, `PositionToken.sol:687`, `:748`
Borrow against a followed position	**Yes** (proven, e2e steps 3, 4, 8)	`LendingPool.sol:249`
Leaderboard	**Partial**	The community market ranks open *positions*: "Top performers" by PnL, with sorts by volume, PnL, holders, notional and newest (`derive.ts:115`). The backend also has `GET /positions/leaderboard` (`discovery.ts:230`). The API has a public portfolio per wallet (`GET /users/:address/portfolio`), but nothing ranks *traders* across their positions.
Copy-execution of a trader's *new* trades	**Roadmap**	A follower holds one position. Later trades by the same creator are not mirrored.
PvP / tournaments	**Roadmap**	Not built

Idea 05 (structured products), honestly. Laxu does not implement options, covered calls or cash-secured puts. What it does build is the kind of wrapper such products are made from: a perp position in a token with fixed on-chain risk parameters (leverage caps, LTV tiers, liquidation rules) and its own isolated lending pool. Option-based structures on the same primitive are a roadmap idea, not part of this submission. Laxu is not entered as a structured-products project.

3. How Laxu uses Perpl's API (reference)

Perpl surface	How Laxu uses it	Code	Notes
REST, public: `/v1/pub/context`, `market-data/ticker`, `/book`, `/candles`, `/funding`	Market list, leverage caps, fees, the min deposit; a cached passthrough for the browser; funding history	`rest.ts:142-161`, `marketData.ts`, `funding.ts:54`	Perpl serves CORS and the market-data socket only for its own origin, so the browser goes through the backend
REST, signed: `trading/wallet`, `positions`, `order-history`, `fills`, `account-history`, `position-history`	Slot balances; the "Fills on Perpl" panel; realised funding per position	`rest.ts:169-221`, `venueFills.ts`	Display and reporting only. Order history lags the socket by about 25 s, so it never decides an order
Trading WebSocket	Sign-in `mt:29`; snapshots `mt:19/23/26`; orders out as `mt:22`; ack `mt:3`; outcomes `mt:24` (orders), `mt:25` (fills), `mt:27` (positions); `mt:21` account; `mt:100` heartbeat; ping `mt:1` / pong `mt:2`	`tradingWs.ts:116-129`, `:261-281`	One socket per slot
Market-data WebSocket	The trade tape (Perpl has no REST trades endpoint). Book and candles come from REST	`marketData.ts:284`	One socket for the backend
Exchange contract, reads	`getPosition`, `getPerpetualInfo`, `isHalted`, `getAccountByAddr`, `getExchangeInfo`, `getMinAccountOpenCNS`	`exchange.ts:54-135`, `PerplReader.sol`	The token reads the mark on-chain itself
Exchange contract, writes	`createAccount`, `depositCollateral`, `withdrawCollateral`, `allowOrderForwarding(true)`	`exchange.ts:171-203`	Sent by slot wallets
AUSD faucet `0xd236…ee6C`	`requestFunds(address)` for test funds, with a transfer fallback	`faucet.ts`	One global 60 s cooldown (measured)
Builder codes	Config and the `bf` field are built behind `PERPL_BUILDER_ENABLED=false`, fee 0	`builder.ts`, `adapter.ts:180`	No builder id issued yet
API-key enrolment	Ed25519 keys per slot wallet; the enrolment script requests scope 3 (read and trade)	`enrollSlotKey.ts`, `slot-provisioning.md`	The script is built and tested against a mock. **Not run against Perpl:** our origin is not whitelisted yet

Field conventions (from `venue/perpl/units.ts`, `requests.ts` and `perpl-findings.md`)

Field	Meaning in Laxu
`rq`	Request id. Strictly above `max(lfr, saved counter, handed out)` for the account (`requests.ts:17`)
`lb`	Last block the order may execute: head + the market's `order_ttl_blocks` (about 20 blocks, about 6 s)
`lv`	Leverage × 100 (`3x` → `300`); Perpl silently clamps anything above the market max
`initial_margin`	Max leverage × 100 (ETH 1200 = 12x), not a fraction
Prices / sizes	Integers scaled by the market's `price_decimals` / `size_decimals` (ETH: 2 and 3, so 2711.29 → `271129`, 0.053 ETH → 53 lots)
Amounts	AUSD base units, 6 decimals (`100000000` = 100 AUSD)
Fees	Micros: `taker_fee: 345` = 0.0345%
Funding `rate`	Micros per funding interval (2,580 s = 43 min on testnet); **positive = longs pay shorts**

Findings we reported or worked around (each verified in `perpl-findings.md`). We share these to help other integrators and Perpl's docs:

1. `initial_margin` is max leverage in hundredths, and an `lv` above the max is accepted and silently clamped. Laxu caps leverage itself before sending.
2. 3 of 22 acknowledged IOCs never reported an outcome: 2 were silently lost and 1 was cut off by a socket close. Laxu decides such orders on-chain (§4).
3. Order history shows an order 22–27 s after the socket does, too late to decide "not placed". Laxu does not use it for that.
4. A successful `t:6` margin top-up sends only a late `st:7` failure frame. Laxu confirms it by the on-chain deposit change.
5. Every size change re-margins the whole position to the new order's `lv`, so buy-ins and redeems send the position's own `lv`.
6. A BTC buy-side IOC with `p: 0` was cancelled unfilled against a full book. Laxu sends an explicit limit at the slippage bound.
7. `pnlCNS` already includes funding (`premiumPnlCNS`), so venue equity = deposit + `pnlCNS`.
8. Fill `at.txid` has no `0x` prefix, `bfa` arrives as `"0"`, and funding is realised into position history (`fnd`) with no `Funding` account events.
9. The minimum account open is 100 AUSD and the minimum deposit is 10 AUSD on testnet (the docs' example differs).

4. Reliable execution

Open-position state machine. `awaiting_payment → payment_received → deposited → order_filled → minted`, with `refunding → refunded` and `failed` (`openPosition.ts:81-88`). Each step writes what it learned to the request row before the next step starts, and checks the row and the venue before acting. A restart resumes; it never repeats a step that moves money (`resumeOpenRequests`, `:397`, run at boot and on every reconciler tick). A five-minute driver lease means only one process drives a request (`:359`). Anything that fails before the fill is refunded. *Proven:* e2e step 1 finished its mint through this resume after a restart, and step 10 refunded a wrong payment in full.

Outcome-unknown rule (the lot rule). Before an order is sent, its `rq`, its `lb` and the on-chain position size are saved. If the socket gives no verdict, the order is decided once the chain is past `lb`. A changed size means it filled, by the difference. An unchanged size means it was not placed, and only then is a *new* `rq` sent. There are at most 3 attempts (`venueOrders.ts:28`, `requests.ts:39`, `adapter.ts:269`). The same order is never sent twice: an order on file is always decided first (`openPosition.ts:657-666`). *Test:* "findOrderOutcome: pending until the chain passes lb, then not_placed (size unchanged) or filled by the lot delta" (`requests.test.ts:22`).

Strictly increasing `rq`, one mover per slot. The `rq` and the slot's counter are saved in one transaction, with a SQL `GREATEST` so the counter never moves backwards (`requests.ts:57-68`). `withSlotLock` serialises every money movement on a slot (`float.ts:68`). Workers run in **one** backend process; the deploy guide forbids a second backend with workers on the same database.

Slot allocator. A slot is reserved when a user is told to pay. Unpaid reservations expire after 15 minutes and are swept before each reservation (`allocator.ts:37`). **Capacity: 5 slots** in the database checked on 10 Oct 2026 (Perpl accounts 824, 841, 1063, 1064 and 1065), so 5 trades can be open at once. The keys for 1063–1065 were made by hand in Perpl's API-key page; programmatic enrolment still waits on Perpl (§10).

Connection handling.

- Reconnect with backoff (1 s, 2 s … 60 s), reset when a wallet snapshot arrives. `1001` reconnects at once, and a `3401` re-signs with a fresh nonce (`tradingWs.ts:497-512`).
- A gap in the `mt:100` heartbeat sequence forces a reconnect for fresh snapshots (`:577-585`).
- The app ping (`mt:1`, 30 s ± 2 s) starts only after sign-in succeeds (`keepAlive.ts:35`). The `ws` library answers protocol pings.
- **The `1008 ping timeout` closes are open, not resolved.** In a 53-minute soak, sockets closed 6–9 times each. In a 30-minute idle run (8 Oct), each socket closed once, which is 2 per hour: at the budget, not under it. Laxu answered all 337 server pings within the same millisecond. Two separate sockets closed within 5 ms of

each other, which points at the server or the network path. An order cut off by a close is decided by the lot rule, and no money was affected.
- **Keep-alive run on the hosted backend: not yet run.** Every run so far was from one PC in Nigeria, with about 0.4 s round trip to Perpl. The order-load run is not run either.

Paced RPC. Monad's public RPC limits requests to 25 per second. Every request goes through one throttle at 12 per second (`RPC_MAX_RPS`), and rate-limit answers are retried with backoff, up to 6 times (`clients.ts:35-66`). Log queries are split into 100-block windows. Writes add 30% to the gas estimate (`GAS_BUFFER_BPS`).

Reconciler and indexer. Every 3 minutes the reconciler compares each position's ledger with Perpl's on-chain position. It flags a slot whose position has vanished with no close in flight, free slots holding money above their reserve, and ledger entries stuck mid-flight. It also resumes open requests (`reconciler.ts:56`). *Proven:* the pass after the e2e close reported drifted 0, unswept 0, stranded 0, missed liquidations 0.

Measured numbers (e2e run, 5 Oct 2026, ETH long $50 at 3x)

Measure	Value	What it includes
Payment block → Perpl deposit	19 s (18:57:26 → 18:57:45 UTC)	Payment detection, slot top-up, `depositCollateral`
Deposit → order filled on-chain	9 s (→ 18:57:54)	Sizing, signing, Perpl's forwarding
Fill → `createPosition`	6 s (→ 18:58:00)	The token's on-chain venue check and mint
Payment → token minted, on-chain	**34 s**	All of the above
Status "minted" in the backend	152 s, an upper bound	Includes a database timeout and a restart (incident 2)
Order outcome over the socket	~1.0–1.5 s after send	19 executed IOCs in the probes

Success and failure counts. All 10 e2e steps passed; 4 of them (1, 2, 5, 9) passed only after a fix or a corrected check. Each incident is listed in `e2e-run.md`. There were 5 open requests in total: 4 minted, and 1 refunded (the deliberate wrong-payment test). In the probes, 3 of 22 acknowledged IOCs never reported an outcome (2 silently lost by the testnet, 1 cut off by a socket close).

5. Risk management and profitability

5.1 Risk management

Position level

- **Max leverage per market** = `min(20, initial_margin / 100)` (`units.ts:195`), checked again before every order (`adapter.ts:152-156`). Caps read from Perpl's context on 5 Oct 2026: **BTC 15x, ETH 12x, SOL 10x, PUMP 5x, MON 3x, ZEC 3x, LIT 3x, NEAR 3x.**
- The factory enforces `MAX_LEVERAGE = 20` (`PositionTokenFactory.sol:22`, `:93`).
- The entry is checked against Perpl within 50 bps (`ENTRY_TOLERANCE_BPS`). Direction and size must match exactly, or `createPosition` reverts (tests: `Venue.js:193-208`).
- **Slippage bound:** each market order carries an explicit limit at mark × (1 ± 1%) (`PERPL_SLIPPAGE_BPS`=100), capped by the market's own maximum (`adapter.ts:59`, `:161`).
- **Perpl's 10 AUSD minimum deposit** is enforced in the ticket and the backend, read from `min_deposit_amount` (commit `ddb1b11`; `openPosition.ts:131-148`).

Lending level (`LendingPool.sol:77-95`, `:477-481`)

Leverage	LTV	Liquidation threshold	Bonus
1–5x	50%	60%	8%
6–10x	40%	50%	10%
11–20x	25%	35%	12%

The close factor is 50%, or 100% below health factor 0.95 or under $50 of collateral. Liquidation is permissionless and **never pauses**: `liquidate()` is not gated by price freshness (tests: `Lending.js:539`, `:637`). Each pool is isolated with a debt ceiling of 10,000 AUSD in the shared vault (`LendingVault.sol:118`). The borrow rate is 10% APR, simple.

Price-safety level

- The mark is read on-chain from Perpl. The operator has no function that sets it (test: "operator has no function that sets the mark price", `Venue.js:291`).
- `isPriceFresh()` needs a mark no older than `MARK_MAX_AGE` = 5 min and a funding report no older than `FUNDING_MAX_AGE` = 2 h (`PositionToken.sol:63-67`, `:470`).
- Stale data blocks new borrowing and collateral withdrawal (`freshOracle`, `LendingPool.sol:203`), but never liquidation.
- *Proven:* borrowing kept working for 10 minutes with the backend down (e2e step 8).

User level

- Per-holder stop-loss and take-profit fire on Perpl's mark. *Proven:* a real stop-loss fired at mark $2706.53 (e2e step 7).
- **Protect this loan:** a Privy signer, limited by policy to `repay()` on one pool up to a per-call cap. See `docs/PRIVY.md`. Server-side QA passed; a full trigger-and-repay cycle ran on 2026-10-13: repay `0x882f5ef4…`.

Operational

- Perpl API keys cannot withdraw. Withdrawals need the slot wallet's EVM key, sent on-chain.
- A float wallet fronts buy-ins and payouts. `recycleFreedMargin` refuses (and alerts on) any withdrawal larger than the payout it funded (`margin.ts:354`).
- Config flags, all off by default: `ENABLE_INDEXER`, `ENABLE_RECONCILER`, `ENABLE_REPORTER`, `ENABLE_LIQUIDATOR`, `ENABLE_PROTECTION` (`env.ts:151-157`), `FAUCET_ENABLED` (`:197`) and `PERPL_BUILDER_ENABLED`. The contracts have no pause function.

What can go wrong	What stops it	Test / evidence
Order sent, outcome lost	Lot rule; new `rq` only when the size did not change	`requests.test.ts:22`; rq 13 in the probes
Leverage above the market max	Capped before sending; factory cap 20	`adapter.ts:152`; leverage experiment
Token that doesn't match Perpl	`createPosition` reverts	`Venue.js:193-208`
Operator pushes a false price	No price setter exists	`Venue.js:291`
Stale mark or funding	Borrowing pauses; liquidation continues	`Lending.js:637`; e2e step 8
Borrow above LTV	Reverts	e2e step 4, tx `0xa2150226…`
Backend down	Mark is on-chain; borrowing continues up to 2 h	e2e step 8
Money swept from holders	Recycle capped at the payout	`margin.ts:354`; e2e incident 5
Residual: funding is operator-reported	Bounded by `FUNDING_MAX_AGE`	Not removed
Residual: bad debt is not absorbed	No insurance fund	Not removed
Residual: contracts are unaudited	none	Not removed

5.2 Profitability: what we can honestly say

Laxu is execution and position infrastructure, not a strategy that picks trades. The edge belongs to the trader who creates the position. Laxu makes their execution safe and their results verifiable. It also gives them a way to earn (2% of each buy-in) and to unlock capital without closing (borrowing).

Our own test trades. These are every Laxu position recorded in the database, read from Perpl's fills and position history by `npm run perpl:report` on 9 Oct 2026. They are test-size trades that show the accounting, **not a track record**.

Position	Market	Side	Lev	Size (opened)	Avg entry	Avg exit	Gross PnL	Fees	Funding	Net (AUSD)
`0xaedd…53f9` (e2e)	ETH	long	3x	0.073	2711.07	2707.68	−0.247230	0.136474	−0.004240	**−0.387944**
`0x96e2…d7fa` (soak)	ETH	long	2x	0.014	2715.92	2697.59	−0.256620	0.026148	−0.013300	**−0.296068**
`0x80a2…cffb` (soak)	BTC	short	2x	0.00045	85763.20	85584.10	+0.080595	0.026602	+0.014535	**+0.068528**
`0x3f0d…7ee9`	ETH	long	3x	0.116	2497.08	open	open	0.099934	0 so far	open
Settled total							**−0.423255**	**0.189224**	**−0.003005**	**−0.615484**

Gross = sell notional − buy notional over the position's fills. Net = gross − fees + funding. Every fee is the 0.0345% taker rate (for example, 0.053 ETH × 2711.29 × 0.000345 = 0.049576). The e2e position's 0.073 ETH includes B's 0.02 ETH buy-in. It was closed in three parts: the redeem, the stop-loss and the final close.

Economics for a trader or follower (arithmetic only)

Item	Formula	Example
Round-trip taker fee	2 × 0.0345% × notional	300*notional*(100 at 3x): **$0.207**
Funding per day	rate × (86,400 / 2,580) × notional	ETH at 20 micros (0.002% per 43 min): 0.067% per day, **$0.20/day** on $300
Creator's buy-in fee	2% × buy-in	20 → * *0.40** (proven); $100 → $2; $1,000 → $20
Borrow cost per day	10% / 365 × debt	$15 → $0.0041; 100 → * *0.0274**; $1,000 → $0.274

The funding rates (BTC 10, ETH 20 micros) are the ones read on 9 Oct 2026 (`perpl-findings.md`). They change every interval.

No strategy bot was run for this submission; a funding-rate demo strategy is on the roadmap.

6. Fit with the judging criteria

Criterion	Our answer in one sentence	Evidence
Reliable execution	Every open is resumable, and an order with an unknown result is decided on-chain and never sent twice.	§4; `requests.test.ts:22`; `e2e-run.md`
Good risk management	Caps per market, an on-chain venue check, a mark the operator cannot set, tiered LTV, and liquidation that never pauses.	§5.1; `Venue.js`, `Lending.js`
Ability to be profitable	Laxu makes a trader's results verifiable and lets them earn 2% of buy-ins	§5.2
Real on-chain activity	69 Perpl orders, 37 fills, 4 tokens, 4 pools, 7 borrows and 1 repay, plus a full verified transaction trail.	§7
Known gaps	1008 socket closes are open; 5 slots; no strategy bot; funding is operator-reported	§10

7. Real on-chain activity

Network: Monad testnet, chain ID 10143. Timeframe: 5 Oct 2026 16:22 UTC (first fill) to 9 Oct 2026 16:13 UTC (last fill). Every hash below was checked with `eth_getTransactionReceipt` on `https://testnet-rpc.monad.xyz` on 9 Oct 2026.

Transactions (e2e run, 5 Oct 2026)

Step	Tx	Who sent it	What it proves
Payment	`0x19d1add7…`	User A	User pays AUSD to the slot
`depositCollateral` (Perpl)	`0x38509ca9…`	Slot wallet `0x27fa…e503`	Funds enter Perpl account 824
Entry order (forwarded)	`0x0f78c601…`	Perpl's forwarder `0x5662…6793` → Exchange	The API order filled on-chain: 0.053 ETH at 2711.29
`createPosition`	`0x0e985052…`	Laxu operator	The token is minted only after it matches Perpl
`createPool`	`0x8de67bc9…`	Laxu operator	An isolated lending pool
`applyFunding`	`0x37e6f74f…`	Laxu operator	The reporter's funding push
`borrow`	`0xb0a458f3…`	User A	Borrowing against the position
Borrow above LTV	`0xa2150226…`	User A	**Reverted** (intended): the LTV limit holds

Step	Tx	Who sent it	What it proves
Buy-in fulfilled	`0xda824be3…`	Laxu operator	B follows A; A gets $0.40
Redeem fulfilled	`0xacf99eca…`	Laxu operator	B exits half at NAV
Stop-loss	`0x93fb5e43…`	Laxu operator	Fired on Perpl's mark
`repay`	`0xef8aadbf…`	User A	Loan repaid
`withdrawCollateral` (pool)	`0xba27321e…`	User A	Shares released
Close fill (forwarded)	`0xa32b2019…`	Perpl's forwarder → Exchange	Position closed on Perpl
`close`	`0xa6b9bb58…`	Laxu operator	Token closed at final NAV
`withdrawCollateral` (Perpl)	`0x370a00e8…`	Slot wallet `0x27fa…e503`	50.086611 AUSD out of Perpl
`settle`	`0xc58c9ebf…`	Laxu operator	$50.086611 for 50 shares
Claim	`0xafb6a727…`	Laxu operator	A is paid
Refund	`0x972409ff…`	Slot wallet `0xef12…2be5`	A wrong payment is returned in full

Activity summary. Command: `cd Backend && npm run perpl:report -- --with-logs`, run on 9 Oct 2026 (chain head 69649262). It sends nothing. Output, with the wallet column dropped (wallets are listed below):

```
Perpl accounts (2 slots in the database)
| Slot   | Account | Status    | Orders | Filled orders | Fills | First fill           |
Last fill            |
| sqmnb8 | 824     | allocated | 29     | 27            | 27    | 2026-10-05T16:22:15Z |
2026-10-09T16:13:24Z |
| 6cc1x5 | 841     | free      | 40     | 10            | 10    | 2026-10-05T16:31:26Z |
2026-10-06T07:58:18Z |
| All    |         |           | 69     | 37            | 37    |                      |
|

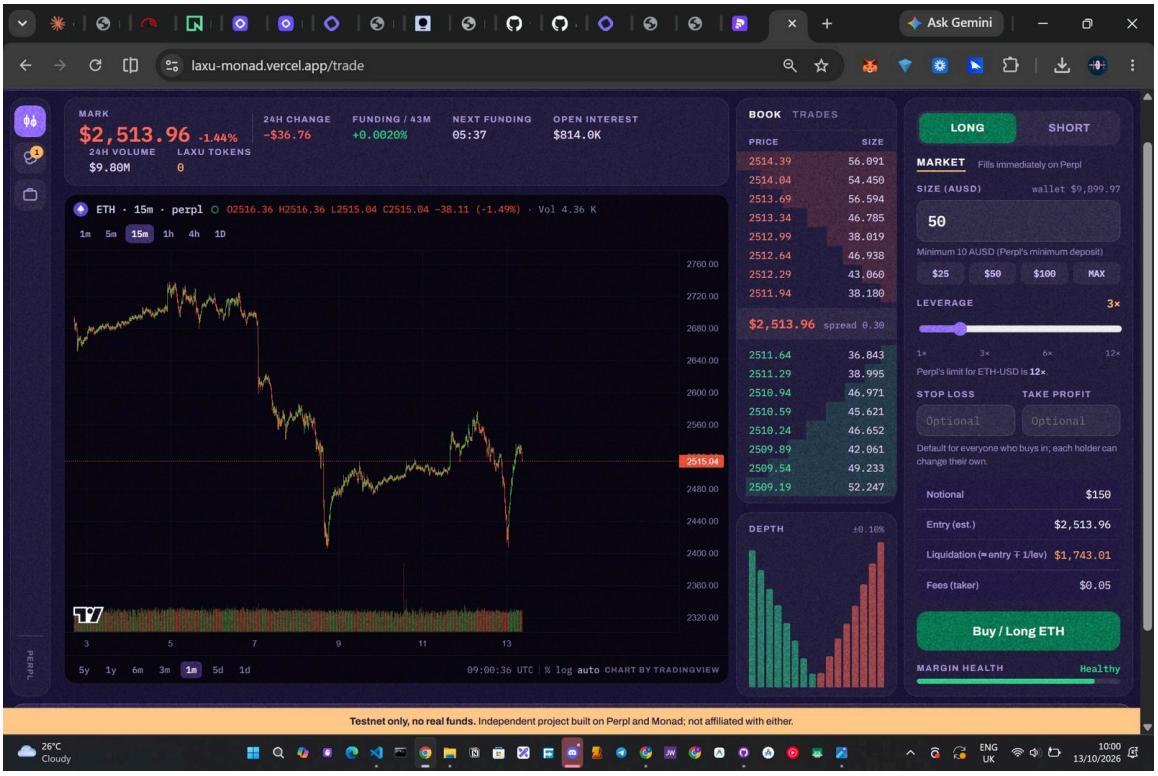
Database
- open requests by status: refunded 1, minted 4
- positions by status: open 1, settled 3
- flows by type: open 4, claim 2, buy_in 1, redeem 1, trigger_exit 1
- NAV report rows (position_reports): 153
- users: 4; distinct creators: 3; distinct flow wallets: 3; distinct holder rows' wallets:
4
- open requests from 2026-10-05T18:57:22.754Z to 2026-10-09T16:12:43.872Z
```

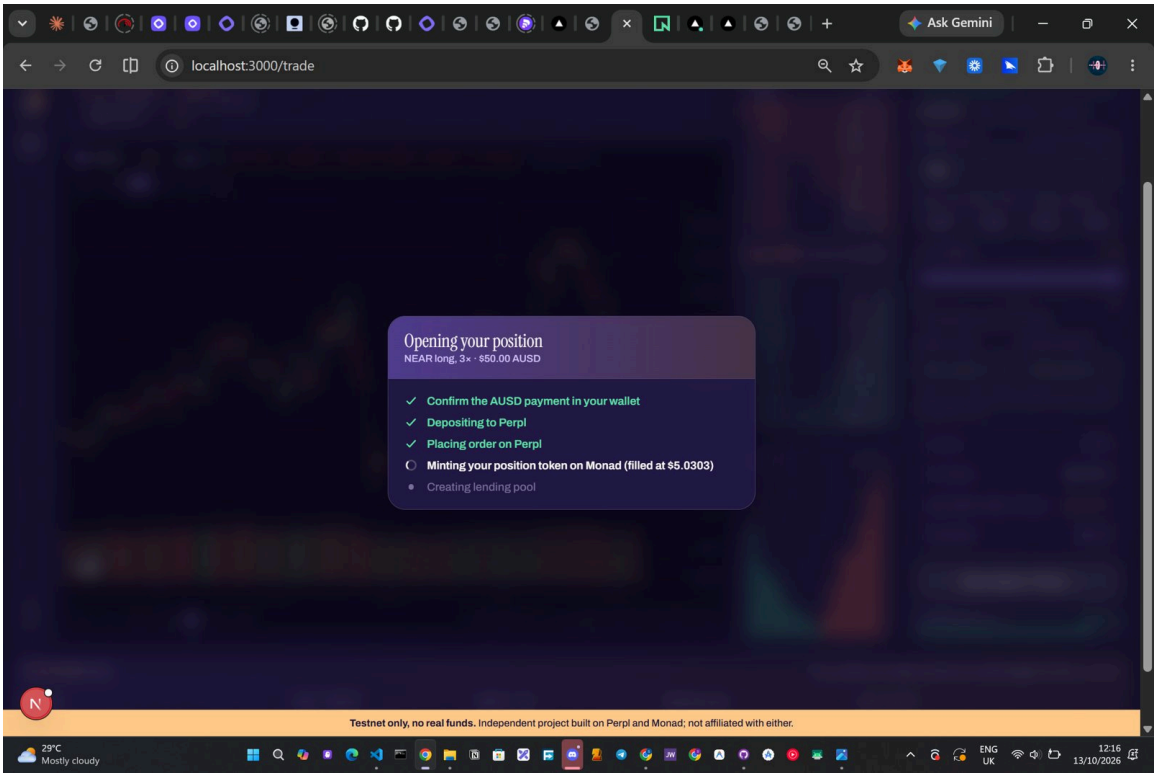
```
On-chain (head 69649262)
- PositionTokenFactory.allPositionsCount() = 4
- LendingPoolFactory.allPoolsCount() = 4
| Pool        | Token       | Blocks scanned    | Deposits | Withdrawals | Borrows |
Repays | Liquidations |
| 0x5566777b… | 0xaedd4332… | 68478350–68495929 | 1        | 1           | 7       | 1
| 0            |
| 0xac3ff65a… | 0x96e279d0… | 68499558–68627585 | 0        | 0           | 0       | 0
| 0            |
| 0x3ab5b933… | 0x80a2fde4… | 68499982–68634331 | 0        | 0           | 0       | 0
| 0            |
| 0xb3a91c5c… | 0x3f0d18db… | 69583601–69649262 | 0        | 0           | 0       | 0
| 0            |
Lending totals: CollateralDeposited 1, Borrowed 7, Repaid 1, CollateralWithdrawn 1;
distinct borrower wallets: 1
```

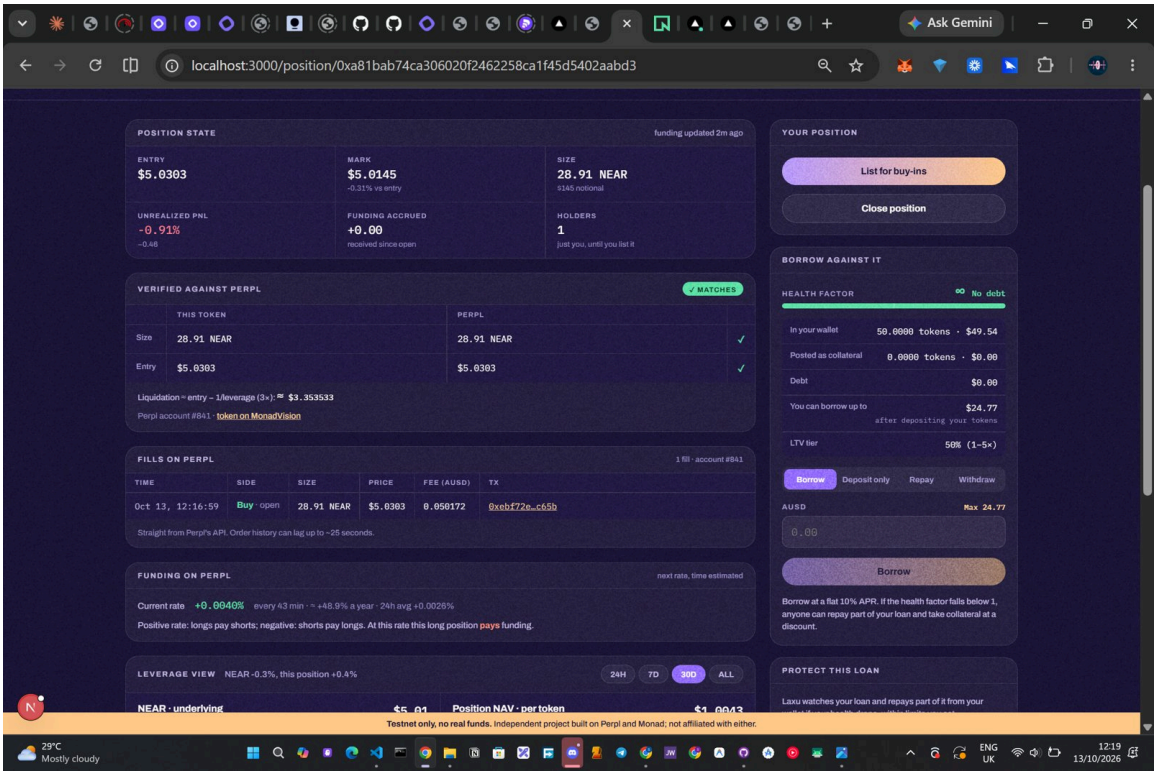
How to read it: the 69 orders include our probe and experiment orders, not only orders for users. Of the 37 fills, 10 belong to the four Laxu positions and 27 to the probes (5 Oct). Most of the 30 unfilled orders on account 841 match the BTC close that was retried during the soak (9 resumes of up to 3 attempts, `e2e-run.md` finding 3). All lending activity is user A's in the e2e run: 1 borrow in step 3 and 6 small borrows in step 8, then 1 repay. These numbers are small, and they are shown as they are.

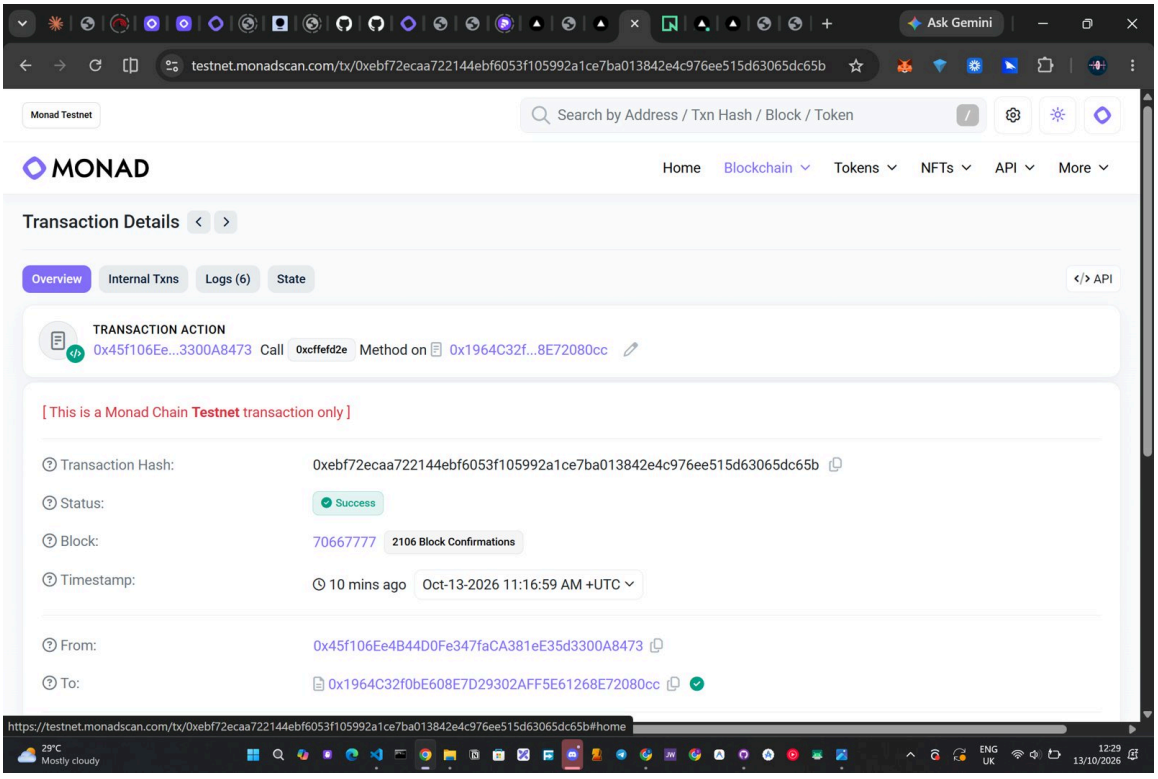
Perpl accounts used (public): slot 1 wallet `0x27faec53e9fdae9e4fac0af5cc4731e77ae8e503`, account **824**; slot 2 wallet `0xef12476197422eab5277866d4170e64974532be5`, account **841**. Perpl Exchange: `0x1964C32f0bE608E7D29302AFF5E61268E72080cc`.

8. Screenshots









Screenshots 02–04 are one trade: a NEAR 3× long, filled on Perpl account 841 on 2026-10-13 at 11:16:59 UTC, forwarded to the Perpl Exchange in tx `0xebf72eca…`.

9. Reproduce it

Env var names (values never shown): `RPC_URL`, `DATABASE_URL`, `OPERATOR_PRIVATE_KEY`, `FLOAT_PRIVATE_KEY`, `SECRET_SLOT_<n>_EVM`, `SECRET_SLOT_<n>_API`, `PERPL_API_KEY_<n>`, `PERPL_API_URL`, `PERPL_WS_URL`, `PERPL_EXCHANGE`, `PERPL_ENROLL_ORIGIN`, and the `ENABLE_*` flags. For e2e users: `E2E_USER_A_KEY` and `E2E_USER_B_KEY`. See `Backend/.env.example`.

```
cd Backend
npm run e2e                          # the 10-step run; needs a backend with every worker on
npm run perpl:report                 # PnL per position + orders/fills per slot (read-only)
npm run perpl:report -- --with-logs  # also counts lending events per pool (a few minutes)
```

What needs Perpl access: a funded Perpl account per slot with order forwarding on, and an Ed25519 API key for each slot wallet. Programmatic enrolment (`npm run slots:enroll`) waits on Perpl whitelisting our origin.

Verify a position against Perpl yourself (no Laxu code). This was run on 9 Oct 2026 for the open token, and both sides agreed at 0.116 ETH, entry 2497.08:

```
// node --input-type=module, with viem installed
import { createPublicClient, http, parseAbi } from "viem";
const c = createPublicClient({ transport: http("https://testnet-rpc.monad.xyz") });
console.log(await c.readContract({
  address: "0x3f0d18db2021255b35885fce4579a492f3a07ee9",
  abi: parseAbi(["function venueDrift() view returns (uint256,uint256,uint256,uint256,bool)"]),
  functionName: "venueDrift",
})); // [ourSize, perplSize, ourEntry, perplEntry, exists]
```

`cast call 0x3f0d18db2021255b35885fce4579a492f3a07ee9 "venueDrift()(uint256,uint256,uint256,uint256,bool)" --rpc-url https://testnet-rpc.monad.xyz`. To read Perpl directly, call `getPosition(uint256 perpId, uint256 accountId)` on the Exchange with `(32, 824)` for ETH on account 824.

10. Status, limits and open items

Item	Status	Detail
Origin whitelisting	Builder registration submitted, reply pending	Needed for key enrolment and direct browser market data; Perpl handles it through the builder registration form

Item	Status	Detail
Builder code	Builder registration submitted, reply pending	Built behind `PERPL_BUILDER_ENABLED=false`; builder fee 0
Delegated accounts	Not built	Per-user accounts via `target_profile`; the owner-side grant is not documented
`1008 ping timeout`	Open	1 close per socket in 30 min idle, 6–9 in a 53-min soak; evidence points outside Laxu; orders are safe through the lot rule
Keep-alive	complete	running
Slots	5	Accounts 824, 841, 1063, 1064 and 1065; 5 trades open at once
Funding	Operator-reported	Bounded by `FUNDING_MAX_AGE` (2 h); the mark is not operator-reported
Strategy bot	None	A funding-rate demo strategy is on the roadmap
Protect this loan	Proven	Full trigger-and-repay cycle: repay `0x882f5ef4…` (`PRIVY.md`)
Testnet resets	Risk	A Perpl testnet reset would affect every open position